

Backing Up & Restoring etcd

A disaster-recovery simulation: what happens when you lose etcd, and how to bring a cluster back from a snapshot

Estimated time: 45-60 minutes

Level: Intermediate — assumes a kubeadm cluster with control-plane SSH access

You will practice: snapshot backups, simulating data loss, full restore, cluster verification

⚠ **Run this on a disposable lab cluster**

This lab intentionally stops etcd and deletes its data directory to simulate a real outage. Never run these destructive steps against a cluster you care about.

Why This Lab Matters

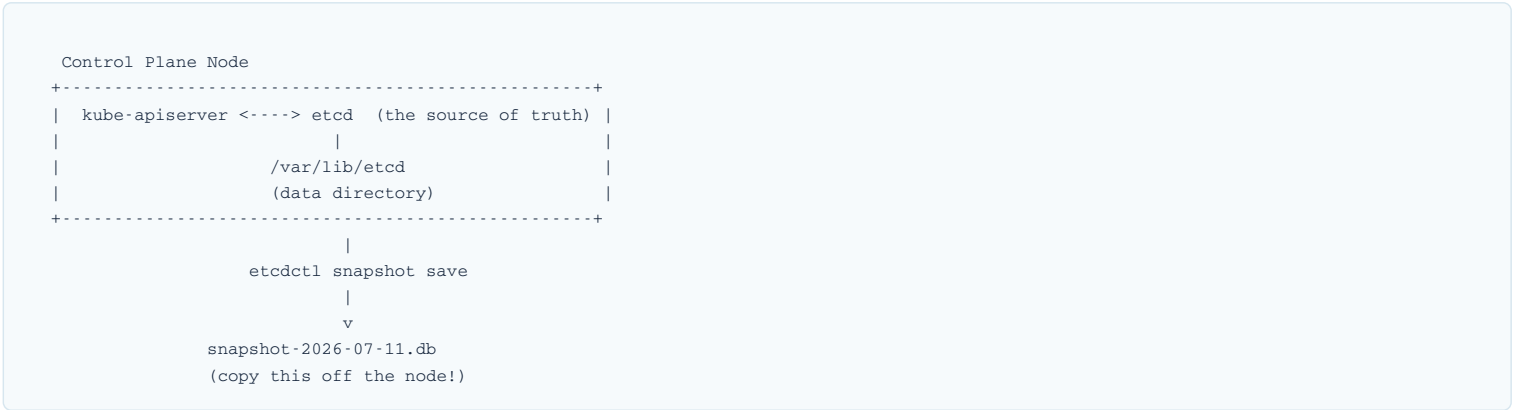
Every object in Kubernetes — every Pod, Deployment, Secret, ConfigMap, and Service — is really just a record stored in **etcd**, the cluster's distributed key-value store. The API Server is the only component that talks to it directly, but etcd is the single source of truth underneath everything. If etcd's data is lost and there is no backup, the cluster's state is lost with it — nodes may still be running containers, but Kubernetes itself no longer knows they exist.

REAL-WORLD INCIDENT

Teams have lost entire production clusters because a disk hosting etcd failed with no recent snapshot to restore from — hours of rebuilding every Deployment, Secret, and Ingress rule by hand followed. A tested backup/restore process turns a multi-day outage into a 10-minute recovery.

What you'll do in this lab

1. Take a proper etcd snapshot while the cluster is healthy
2. Deploy some "real" workloads to represent production data
3. **Simulate a disaster** — stop etcd and destroy its data directory
4. Confirm the cluster is broken
5. Restore etcd from the snapshot and bring the cluster back to life
6. Verify every workload came back exactly as it was



Prerequisites

- A kubeadm-created cluster (this lab assumes etcd runs as a static Pod on the control-plane node)
- SSH / terminal access to the control-plane node with sudo rights
- `etcdctl` installed (or run it via the etcd static Pod — both are shown below)
- `kubectl` configured against the cluster

Installing `etcdctl`

`kubectl` only ever talks to the **Kubernetes API server** — it has no concept of etcd at all. Taking a snapshot, checking etcd's health, or restoring data are operations that happen one layer below the API server, directly against the datastore. **etcdctl** is the official command-line client for etcd, and it's the only supported way to do any of that. Without it, there's no way to back up or restore a cluster's state.

Option A — Package manager (Ubuntu/Debian)

```
$ sudo apt-get update
$ sudo apt-get install -y etcd-client

$ etcdctl version
# etcdctl version: 3.5.9
# API version: 3.5
```

Option B — Download the official binary

```
$ ETCD_VER=v3.5.13
$ curl -L https://github.com/etcd-io/etcd/releases/download/\
${ETCD_VER}/etcd-${ETCD_VER}-linux-amd64.tar.gz \
-o /tmp/etcd.tar.gz
$ tar xzvf /tmp/etcd.tar.gz -C /tmp
$ sudo mv /tmp/etcd-${ETCD_VER}-linux-amd64/etcdctl \
/usr/local/bin/

$ etcdctl version
```

MATCH YOUR SERVER VERSION

Install an `etcdctl` version that matches (or is close to) the etcd version your cluster actually runs — mismatched major versions can behave unpredictably against snapshot/restore operations. Check the server's version first:

```
$ kubectl exec -n kube-system etcd-control-plane -- etcd --version
```

NO INSTALL ACCESS? USE THE ETCD POD DIRECTLY

If you can't install anything on the node, every command in this lab can also run *inside* the existing etcd static Pod, which already ships with a matching `etcdctl`:

```
$ kubectl exec -n kube-system etcd-control-plane -- \
  etcdctl snapshot save /tmp/snapshot.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
```

This lab uses the standalone-binary form for clarity, but either approach works.

FINDING YOUR ETCD CERTS

On a standard kubeadm cluster, the etcd client certificates live at `/etc/kubernetes/pki/etcd/` and the API server always talks to etcd at `https://127.0.0.1:2379`. The commands below assume these default paths — adjust them if your cluster was bootstrapped differently.

PHASE 1 — Take a Healthy Backup

1 Confirm etcd is healthy

SSH into the control-plane node and check that the etcd static Pod is running:

```
# On the control-plane node
$ kubectl get pods -n kube-system | grep etcd

# Expected:
etcd-control-plane   1/1   Running   0   4d
```

Check cluster health directly with `etcdctl`:

```
$ sudo ETCDCTL_API=3 etcdctl endpoint health \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# Expected:
https://127.0.0.1:2379 is healthy: successfully committed proposal: took = 12.3ms
```

2 Create some "production" workloads

So the simulation feels real, create a few objects that represent live application state:

```
$ kubectl create deployment demo-app --image=nginx --replicas=3
$ kubectl create configmap important-config --from-literal=release=v2.4.1
$ kubectl create secret generic db-credentials --from-literal=password=SuperSecret123

$ kubectl get deployments,configmaps,secrets
```

3 Take the snapshot

```
# Create a folder to hold backups
$ sudo mkdir -p /opt/etcd-backup

# Save a snapshot
$ sudo ETCDCTL_API=3 etcdctl snapshot save \
  /opt/etcd-backup/snapshot-$(date +%Y%m%d-%H%M%S).db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# Expected:
Snapshot saved at /opt/etcd-backup/snapshot-20260711-101500.db
```

4 Verify the snapshot is valid

```
$ sudo ETCDCTL_API=3 etcdctl snapshot status \
/opt/etcd-backup/snapshot-20260711-101500.db --write-out=table
```

```
# Expected: a table showing hash, revision, total keys, and size
+-----+-----+-----+-----+
| HASH | REVISION | TOTAL KEYS | TOTAL SIZE |
+-----+-----+-----+-----+
| 4a7f2c1e | 1842 | 612 | 2.9 MB |
+-----+-----+-----+-----+
```

PRODUCTION BEST PRACTICE

Never leave the only copy of a snapshot on the same disk as etcd itself. Copy it to another node, object storage (S3/GCS), or a backup server immediately — that's the whole point of a backup.

```
# Copy the snapshot off the node, e.g. to another machine
$ scp /opt/etcd-backup/snapshot-20260711-101500.db backup-server:/backups/etcd/
```

PHASE 2 — Simulate the Disaster

Now we pretend to be an unlucky on-call engineer: the disk holding etcd's data has failed, or someone ran a destructive command. We'll simulate this by stopping etcd and deleting its data directory.

POINT OF NO RETURN (IN THIS LAB)

The steps below destroy the cluster's current state. That's intentional — it's what we're practicing recovering from. Confirm you're on a disposable lab cluster before continuing.

5 Stop the kubelet so the static Pod stops managing etcd

```
$ sudo systemctl stop kubelet

# The etcd static Pod will disappear once kubelet stops watching
# /etc/kubernetes/manifests/etcd.yaml
```

6 Destroy the etcd data directory

```
# Simulate a corrupted / failed disk by moving the data dir away
$ sudo mv /var/lib/etcd /var/lib/etcd-LOST-$(date +%s)

# etcd now has nothing to serve from
```

7 Confirm the cluster is broken

```
$ kubectl get nodes

# Expected:
The connection to the server 10.0.0.10:6443 was refused - did you specify the right host or port?
```

WHAT JUST HAPPENED

The API server depends entirely on etcd. With etcd's data gone, the API server can't start correctly, so `kubectl` can't reach the cluster at all — every Deployment, Secret, and ConfigMap we created is currently unrecoverable except from the snapshot.

PHASE 3 — Restore From Snapshot

8 Restore the snapshot into a new data directory

`etcdctl snapshot restore` never writes into an existing directory — it always creates a fresh one, which we then swap into place.

```
$ sudo ETCDCTL_API=3 etcdctl snapshot restore \
/opt/etcd-backup/snapshot-20260711-101500.db \
--data-dir=/var/lib/etcd-restored

# Expected:
2026-07-11T10:22:01Z info restored snapshot {"path": ".../snapshot-...db",
"wal-dir": "/var/lib/etcd-restored/member/wal",
"data-dir": "/var/lib/etcd-restored", "snap-dir": "/var/lib/etcd-restored/member/snap"}
```

9 Swap the restored data into place

```
$ sudo mv /var/lib/etcd-restored /var/lib/etcd
$ sudo chown -R root:root /var/lib/etcd
```

10 Start kubelet again

```
$ sudo systemctl start kubelet

# kubelet re-reads /etc/kubernetes/manifests/etcd.yaml and
# recreates the etcd static Pod, now pointing at the restored data
```

11 Verify the cluster is back

```
$ kubectl get nodes

# Expected — API server responds again:
NAME                STATUS    ROLES    AGE   VERSION
control-plane       Ready     control-plane   4d    v1.30.2
worker-1            Ready                    4d    v1.30.2
```

12 Confirm the "production" data survived

```
$ kubectl get deployments,configmaps,secrets

# Expected — everything from before the disaster is present:
NAME                                READY   UP-TO-DATE   AVAILABLE
deployment.apps/demo-app           3/3     3             3

NAME                                DATA
configmap/important-config         1

NAME                                TYPE
secret/db-credentials              Opaque
```

SUCCESS

The Deployment, ConfigMap, and Secret all reappeared exactly as they were at the moment the snapshot was taken — because that's literally what a restore does: it replays etcd back to that point in time. Any changes made *after* the snapshot (and before the disaster) would be lost, which is why backup frequency matters.

Verification Checklist

Check	Command	Expected result
API server reachable	<code>kubectl get nodes</code>	All nodes listed as <code>Ready</code>
etcd Pod healthy	<code>kubectl get pods -n kube-system</code>	<code>etcd-*</code> is <code>1/1 Running</code>
Workloads restored	<code>kubectl get deployments</code>	<code>demo-app</code> present, 3/3 ready
Config data restored	<code>kubectl get configmap important-config -o yaml</code>	<code>release: v2.4.1</code> present
Secrets restored	<code>kubectl get secret db-credentials</code>	Secret exists, type <code>Opaque</code>

Cleanup

```
$ kubectl delete deployment demo-app
$ kubectl delete configmap important-config
$ kubectl delete secret db-credentials
$ sudo rm -rf /var/lib/etcd-LOST-*
```

Production Best Practices

- **Automate snapshots** — run `etcdctl snapshot save` on a schedule (e.g. a CronJob or systemd timer), not manually.
- **Store backups off-cluster** — object storage (S3/GCS/Azure Blob) or a separate backup server, never only on the control-plane disk.
- **Encrypt snapshots at rest** — a snapshot contains everything, including raw Secret data.
- **Test restores regularly** — a backup you've never restored from is a guess, not a plan. Run this exact drill periodically.
- **Document your RTO/RPO** — know how old your last snapshot can be (Recovery Point Objective) and how fast you can restore (Recovery Time Objective).
- **Run etcd as a multi-member cluster (3 or 5 nodes)** in real production — this lab simulates total data loss on a single-node etcd for

teaching purposes; a healthy multi-member cluster tolerates losing one member without any downtime at all.

REAL-WORLD ANALOGY

Think of the etcd snapshot like a save file in a video game. If your character dies (the cluster goes down), you don't have to replay the whole game from scratch — you reload the last save point. The only data you lose is whatever happened after that last save, which is exactly why frequent, automated saves (backups) matter.